

[← Go Back](#)

Library

Recents viewed

WiFiNINA

Arduino SigFox for
MKRFox1200

autowp-mcp2515

Arduino_MKRENV

Arduino_MKRGPS

Arduino_APDS9960

Arduino_MKRIoTCarrier

[Home](#) / [Programming](#) / [Library](#) / **WiFiNINA**

COMMUNICATION

WiFiNINA

 **ARDUINO**  LGPL-2.1  V1.9.1  Arduino  20/03/2025 [Arduino <info@arduino.cc>](mailto:info@arduino.cc) <http://www.arduino.cc/en/Ref...>  info@arduino.cc

Enables network connection (local and Internet) with the Arduino MKR WiFi 1010, Arduino MKR VIDOR 4000, Arduino Uno WiFi Rev.2 and Nano 33 IoT.

With this library you can instantiate Servers, Clients and send/receive UDP packets through WiFi. The board can connect either to open or encrypted networks (WEP, WPA). The IP address can be assigned statically or through a DHCP. The library can also manage DNS.

GO TO REPOSITORY

Usage/Examples

Compatibility

Releases

This library allows you to use the Arduino Uno WiFi Rev.2, Arduino Nano 33 IoT, Arduino MKR 1010 and Arduino MKR VIDOR 4000 WiFi capabilities. It can serve as either a server accepting incoming connections or a client making outgoing ones. The library supports WEP, WPA2 Personal and WPA2 Enterprise encryptions. This library supports all the same methods of the original WiFi library plus the `connectSSL()`. The WiFiNINA library is very similar to the Ethernet and the WiFi library, and many of the function calls are the same.

To use this library:

```
1 #include <SPI.h>
2 #include <WiFiNINA.h>
```

Examples

Several examples for the **WiFiNINA** library are available from the [Examples from Libraries page](#).

ON THIS PAGE

[Usage/Examples](#)[Compatibility](#)[Releases](#)[WiFi Class](#) +[Client Class](#) +[Server Class](#) +[UDP Class](#) +[WiFi Class](#)**[Client Class](#)**[Server Class](#)[UDP Class](#)[Help](#)

Description

Client is the base class for all WiFi client based calls. It is not called directly, but invoked whenever you use a function that relies on it.

WiFiClient()

Description

Creates a client that can connect to to a specified internet IP address and port as defined in client.connect().

Syntax

```
1 WiFiClient client;
```

Parameters

client : the named client to refer to

Returns

None

Example

```
1 #include <SPI.h>
2 #include <Wi-FiNINA.h>
3
4 char ssid[] = "myNetwork";
5 char pass[] = "myPassword";
6
7 int status = WL_IDLE_STATUS;
8 IPAddress server(74,125,115,1
9
10 // Initialize the client libr
11 WiFiClient client;
12
13 void setup() {
14   Serial.begin(9600);
15   Serial.println("Attempting
16   Serial.print("SSID: ");
17   Serial.println(ssid);
18
19   status = WiFi.begin(ssid, p
20   if ( status != WL_CONNECTED
21     Serial.println("Couldn't
22     // don't do anything else
23     while(true);
24   }
25   else {
26     Serial.println("Connected
27     Serial.println("\nStartin
28     // if you get a connectio
29     if (client.connect(server
```

Wi-FiSSLClient

Description

This class allows to create a client that always connects in SSL to the specified IP address and port, even if `client.connect()` is used instead of `client.connectSSL()`. This is useful if you have a library that accepts only plain Client, but you want to force it to use SSL, keeping the same method names of the non SSL client.

Syntax

```
1 Wi-FiNINASSLClient client;
```

Parameters

client : the named client to refer to

Return

None

Example

```
1  /*
2  This example creates a client
3  data using always SSL.
4
5  It is compatible with the met
6  connections, like client.conn
7
8  Written by Arturo Guadalupi
9  last revision November 2015
10
11 */
12
13 #include <SPI.h>
14 #include <WiFiNINA.h>
15
16 #include "arduino_secrets.h"
17 //please enter your sens
18 char ssid[] = SECRET_SSID;
19 char pass[] = SECRET_PASS;
20 int keyIndex = 0;
21
22 int status = WL_IDLE_STATUS;
23 // if you don't want to use D
24 // use the numeric IP instead
25 //IPAddress server(74,125,232
26 char server[] = "www.google.c
27
28 // Initialize the Ethernet cl
29 // with the IP address and ss
```

client.connected()

Description

Whether or not the client is connected. Note that a client is considered connected if the connection has been closed but there is still unread data.

Syntax

```
1 client.connected()
```

Parameters

None

Returns

Returns true if the client is connected,

Example

```
1 #include <SPI.h>
2 #include <Wi-FiNINA.h>
3
4 char ssid[] = "myNetwork";
5 char pass[] = "myPassword";
6
7 int status = WL_IDLE_STATUS;
8 IPAddress server(74,125,115,1
9
10 // Initialize the client libr
11 WiFiClient client;
12
13 void setup() {
14   Serial.begin(9600);
15   Serial.println("Attempting
16   Serial.print("SSID: ");
17   Serial.println(ssid);
18
19   status = WiFi.begin(ssid, p
20   if ( status != WL_CONNECTED
21     Serial.println("Couldn't
22     // don't do anything else
23     while(true);
24   }
25   else {
26     Serial.println("Connected
27     Serial.println("\nStartin
28     // if you get a connectio
29     if (!client.connect(server
```

▼ **client.connect()**

Description

Connect to the IP address and port specified in the constructor. The return value indicates success or failure. `connect()` also supports DNS lookups when using a domain name (e.g., `google.com`).

Syntax

```
1 client.connect(ip, port)
2 client.connect(URL, port)
```

Parameters

`ip`: the IP address that the client will connect to (array of 4 bytes)

`URL`: the domain name the client will connect to (string e.g., "arduino.cc")

`port`: the port that the client will connect to (int)

Returns

Returns true if the connection succeeds, false if not.

Example

```
1 #include <SPI.h>
2 #include <Wi-FiNINA.h>
3
4 char ssid[] = "myNetwork";
5 char pass[] = "myPassword";
6
7 int status = WL_IDLE_STATUS;
8 char servername[]="google.com
9
10 WiFiClient client;
11
12 void setup() {
13   Serial.begin(9600);
14   Serial.println("Attempting
15   Serial.print("SSID: ");
16   Serial.println(ssid);
17
18   status = WiFi.begin(ssid, p
19   if ( status != WL_CONNECTED
20     Serial.println("Couldn't
21     // don't do anything else
22     while(true);
23   }
24   else {
25     Serial.println("Connected
26     Serial.println("\nStartin
27     // if you get a connectio
28     if (client.connect(server
29     Serial.println("connect
```

▼ client.connectSSL()

Description

Connect to the IP address and port specified in the constructor using the SSL protocol. The method connectSSL is required when the server provides only HTTPS connections. Before using this method, it is required to load the SSL certificate used by the server into the Arduino WiFi module . The boards come already loaded with certificates and it should be ready to use. To change or upload new SSL certificates you should follow the procedures that will be made available. connectSSL() also supports DNS lookups when using a domain name (e.g., google.com).

Syntax

```
1 client.connectSSL(ip, port)
2 client.connectSSL(URL, port)
```

ip: the IP address that the client will connect to (array of 4 bytes)

URL: the domain name the client will connect to (string e.g., "arduino.cc")

port: the port that the client will connect to (int)

Returns

Returns true if the connection succeeds, false if not.

Example

```

1  ...
2
3  /*
4   Web client
5
6   This sketch connects to a we
7   using a WiFi board.
8
9   This example is written for
10  WEP or WPA, change the WiFi.
11
12  Circuit:
13   * WiFiNINA supported board
14
15   created 13 July 2010
16   by dlf (Metodo2 srl)
17   modified 31 May 2012
18   by Tom Igoe
19   */
20
21
22  #include <SPI.h>
23  #include <WiFiNINA.h>
24
25  char ssid[] = "yourNetwork";
26  char pass[] = "secretPassword";
27  int keyIndex = 0;
28
29  // ...

```

▼ client.status()

Description

Return Connection status.

Syntax

```

1  client.status()

```

Parameters

None

Returns

The client connection status

Example

```
1  ...
2
3  void setup() {
4
5      Serial.begin(9600);
6      while (!Serial) {
7          ; }
8      if (WiFi.status() == WL_NO_
9          Serial.println("Commu
10             while (true);
11      }
12
13      while (status != WL_CONNEC
14          Serial.print("Attempt
15          Serial.println(ssid);
16          status = WiFi
17          delay(10000);
18      }
19      Serial.println("Connected t
20      printWifiStatus();
21
22      Serial.println("\nStarting
23      IPAddress result;
24      int err = WiFi.hostByName(s
25      if (err == 1) {
26          Serial.print("IP addr
27          Serial.println(result
28      } else {
29          Serial.print("Error: "
```

▼ client.write()

Description

Write data to all the clients connected to a server.

Syntax

```
1  client.write(data)
2  client.write(buffer, size);
```

data: the outgoing byte
buffer: the outgoing message
size: the size of the buffer

Returns

The number of bytes written. It is not necessary to read this.

▼ client.print()

Description

Print data to the server that a client is connected to. Prints numbers as a sequence of digits, each an ASCII character (e.g. the number 123 is sent as the three characters '1', '2', '3').

Syntax

```
1 client.print(data)
2 client.print(data, BASE)
```

Parameters

data: the data to print (char, byte, int, long, or string)

BASE (optional): the base in which to print numbers: DEC for decimal (base 10), OCT for octal (base 8), HEX for hexadecimal (base 16).

Returns

byte : returns the number of bytes written, though reading that number is optional

▼ client.println()

Description

Print data, followed by a carriage return and newline, to the server a client is connected to. Prints numbers as a sequence of digits, each an ASCII character (e.g. the number 123 is sent as the three characters '1', '2',

Syntax

```
1 client.println()  
2 client.println(data)  
3 client.print(data, BASE)
```

Parameters

data (optional): the data to print (char, byte, int, long, or string)

BASE (optional): the base in which to print numbers: DEC for decimal (base 10), OCT for octal (base 8), HEX for hexadecimal (base 16).

Returns

byte: return the number of bytes written, though reading that number is optional

▼ client.available()

Description

Returns the number of bytes available for reading (that is, the amount of data that has been written to the client by the server it is connected to).

available() inherits from the Stream utility class.

Syntax

```
1 client.available()
```

Parameters

None

Returns

The number of bytes available.

Example

```
1 #include <SPI.h>
2 #include <Wi-FiNINA.h>
3
4 char ssid[] = "myNetwork";
5 char pass[] = "myPassword";
6
7 int status = WL_IDLE_STATUS;
8 char servername[]="google.com
9
10 WiFiClient client;
11
12 void setup() {
13   Serial.begin(9600);
14   Serial.println("Attempting
15   Serial.print("SSID: ");
16   Serial.println(ssid);
17
18   status = WiFi.begin(ssid, p
19   if ( status != WL_CONNECTED
20     Serial.println("Couldn't
21     // don't do anything else
22     while(true);
23   }
24   else {
25     Serial.println("Connected
26     Serial.println("\nStartin
27     // if you get a connectio
28     if (client.connect(server
29     Serial.println("connect
```

▼ client.peek()

Description

Read a byte from the file without advancing to the next one. That is, successive calls to peek() will return the same value, as will the next call to read().

This function inherited from the Stream class. See the Stream class main page for more information.

Syntax

```
1 client.peek()
```

Parameters

None

Returns

b: the next byte or character

Example

```
1  ...
2
3  #include <SPI.h>
4  #include <WiFiNINA.h>
5
6  #include "arduino_secrets.h"
7  char ssid[] = SECRET_SSID;
8  char pass[] = SECRET_PASS;
9  int keyIndex = 0;
10
11 int status = WL_IDLE_STATUS;
12 char server[] = "www.google.c
13
14
15 WiFiClient client;
16
17 void setup() {
18   //Initialize serial and wai
19   Serial.begin(9600);
20   while (!Serial) {
21       ;   }
22
23   if (WiFi.status() == WL_NO_
24       Serial.println("Commu
25       while (true);
26   }
27
28   while (status != WL_CONNECT
29       Serial.print("Attempt
```

▼ client.read()

Description

Reads data from the client. If no arguments are given, it will return the next character in the buffer.

Syntax

```
1 client.read()
2 client.read(buffer, size);
```

Parameters

buffer: buffer to hold incoming packets (char*)
len: maximum size of the buffer (int)

Returns

b: the next character in the buffer
(char)

size: the size of the data

-1: if no data is available

▼ `client.flush()`

Description

Clears the buffer once all outgoing characters have been sent.

`flush()` inherits from the [Stream](#) utility class.

Syntax

```
1 client.flush()
```

Parameters

None

Returns

None

▼ `client.stop()`

Description

Disconnect from the server.

Syntax

```
1 client.stop()
```

Parameters

None

Returns

None

▼ **client.remoteIP()****Description**

Gets the IP address of the remote connection.

Syntax

```
1 client.remoteIP()
```

Parameters

None

Returns

The IP address of the host the client is connected to

▼ **client.remotePort()****Description**

Gets the port number of the remote connection.

Syntax

```
1 client.remotePort()
```

Parameters

None

Returns

The port of the remote host that the client is connected to

Was this article helpful?

Connect and Contribute

[Project Hub](#)

[GitHub Repository](#)

[Forum](#)

[Product Compliance](#)

[Help Center](#)

[Trademarks & Licensing](#)